

PgBigData — Public-Data API Ingestion → PostgreSQL (JSONB + typed columns)

A small but production-shaped pipeline that ingests **U.S. Census American Community Survey (ACS)** data from the Census REST API and lands it in PostgreSQL using a deliberate **JSONB-vs-typed-column** storage model, ready to expose to a headless CMS (Payload).

It's built to demonstrate the patterns that matter for ongoing public-data ingestion work:

Concern	Where it lives
Pagination / chunking large pulls	<code>census/client.py</code> — fan-out by parent geography
Rate limiting + retries w/ backoff & jitter	<code>census/client.py</code>
Auth	optional API key, <code>config.py</code>
Idempotent / incremental loads	upsert on natural key + <code>load_runs</code> watermark, <code>census/loader.py</code>
JSONB vs typed columns	<code>census/variables.py</code> + <code>sql/001_schema.sql</code> , rationale in <code>docs/DESIGN.md</code>
GIN indexing of payloads	<code>sql/001_schema.sql</code>
CMS-facing contract	views in <code>sql/002_views.sql</code> , mapping in <code>docs/PAYLOAD.md</code>
Pure, testable transforms	<code>census/transform.py</code> + <code>tests/</code>

Why Census ACS

It's the most accessible of the role's three core sources (ACS, voter files, CCES) — fully public, gated only by a free/instant API key — so this sample is runnable end-to-end. The ingestion patterns (chunked fan-out, idempotent upserts, JSONB+typed storage) transfer directly to voter files and CCES survey/crosswalk data; only the client and promotion map change.

Quickstart

```

# 1. Start Postgres (or point DATABASE_URL at your own)
make db-up

# 2. Install + configure
make install
cp .env.example .env          # then add your free CENSUS_API_KEY (required)
# grab one instantly at https://api.census.gov/data/key_signup.html
export $(grep -v '^#' .env | xargs) # or use direnv / your shell of choice

# 3. Create schema + views
make init-db

# 4. Ingest. County level is a single API call (~3k rows):
make ingest-county

# Tract level fans out to one throttled, retried request per state (~85k rows):
make ingest-tract

# PUMS person-level microdata (chunked per state; weighted records):
python -m pgbigdata.cli ingest-pums --year 2022          # WY, VT, DC (fast)
python -m pgbigdata.cli ingest-pums --year 2022 --states 06,48 # CA, TX
python -m pgbigdata.cli ingest-pums --year 2022 --all-states # nation

# CCES survey microdata (bulk file from Harvard Dataverse, ~60k respondents):
python -m pgbigdata.cli ingest-cces --year 2018
# respondents carry county FIPS -> join to ACS via the v_cces_acs_county view

# 5. See what ran
make status

```

Unit tests need no database or network:

```
make test
```

Explore it visually

A Streamlit dashboard provides point-and-click slice-and-dice over the loaded data — filter by geography/state/population, rank, chart distributions, compare income vs. home value, and pivot by state:

```

pip install -e ".[dashboard]"
make dashboard          # opens http://localhost:8501

```

What you get after a county load

```
-- Typed columns: indexed, joinable, CMS-ready
SELECT name, total_population, median_household_income
FROM v_acs_latest
WHERE geography = 'county'
ORDER BY median_household_income DESC NULLS LAST
LIMIT 5;

-- Un-promoted fields, straight from JSONB, no migration needed
SELECT name, raw -> 'B25001_001E' AS housing_units
FROM acs_observations
WHERE geography = 'county' LIMIT 5;

-- JSONB containment, served by the GIN index
SELECT count(*) FROM acs_observations
WHERE raw @> '{"state": "06"}'; -- all California rows
```

Design docs

- docs/DESIGN.md — the JSONB-vs-typed-column decision framework, indexing, idempotency.
- docs/PAYLOAD.md — how these tables/views map to Payload collections.

Project layout

src/pgbigdata/	
config.py	env-driven config
db.py	psycopg connection + SQL runner
cli.py	init-db ingest-acс status
census/	
client.py	API client: retries, throttle, geography chunking
geography.py	geography levels + GEOID construction
variables.py	variable catalog + the promotion map
transform.py	pure row -> record (typed cols + JSONB)
loader.py	idempotent batched upsert + run audit log
sql/	
001_schema.sql	tables, typed cols, JSONB, GIN + btree indexes
002_views.sql	CMS-facing views

tests/

test_transform.py pure-function tests, no infra